Aegis-IR: Eliminating V8 Garbage Collection Pressure
through Deterministic Linear Memory Architecture

Rudranarayan Jena
Founder of Voxion Labs - DY Patil International University - Pune, India

ABSTRACT

Aegis-IR is a browser-native information retrieval architecture designed to reduce main-thread stutter caused by garbage-collection pressure in allocation-heavy Vanilla JavaScript search. The system compiles an object-oriented C++ TF-IDF ranking kernel to WebAssembly and stores its retrieval structures inside deterministic linear memory. During ranking, the engine traverses continuous numeric buffers rather than constructing transient JavaScript arrays, maps, and result objects. This isolates the critical scoring path from V8 heap allocation and models 0 ms garbage-collection pause behavior during query execution. The paper presents the memory problem, the Aegis-IR architecture, the continuous-array scoring model, and browser-side telemetry for measuring thread blocking and allocation pressure.

Primary claim

For browser-native search, the decisive systems boundary is not only the algorithm. It is the memory substrate: managed JavaScript heap allocation versus deterministic WebAssembly linear memory.

CONTRIBUTIONS

- Defines V8 garbage-collection stutter as a first-class failure mode for interactive client-side search.
- Introduces a zero-backend C++ WebAssembly retrieval kernel using contiguous numeric memory for TF-IDF scoring.
- Separates UI orchestration from retrieval execution through a small explicit WebAssembly ABI.
- Provides a telemetry model for thread blocking, heap allocation pressure, and deterministic linear-memory size.

PAPER LAYOUT

Page	Focus	Primary Artifact
1	Abstract and claims	Contribution map
2	V8 GC bottleneck	Memory-path diagram and allocation table
3	Aegis-IR architecture	System tree and TF-IDF array model
4	Telemetry and benchmarks	Stacked graph and execution table
5	Discussion	Limitations, future work, conclusion, references

THE BROWSER SEARCH MEMORY PROBLEM

Vanilla JavaScript search implementations are easy to build because the language makes arrays, maps, closures, and strings cheap to express. Under repeated interactive queries, those constructs become a runtime liability. Every keystroke can allocate a new graph of short-lived objects. V8 can reclaim those objects, but collection timing is not controlled by the search algorithm. When garbage collection overlaps with input handling or rendering, the user sees a stalled interface.

ALLOCATION HOTSPOTS IN VANILLA JAVASCRIPT SEARCH

Allocation Source	Typical Object Shape	Stutter Risk
Tokenization	arrays of strings	high churn per keystroke
Term counting	maps and boxed counters	temporary object graph
Scoring	per-document score records	linear growth with corpus
Sorting	result objects and comparators	heap pressure near render
Highlighting	substring copies	extra allocations after ranking

RUNTIME PATH DIAGRAM

Observed failure mode

The search logic may be locally correct, but the managed heap introduces non-deterministic pauses. Aegis-IR therefore treats memory allocation behavior as part of the algorithmic design surface.

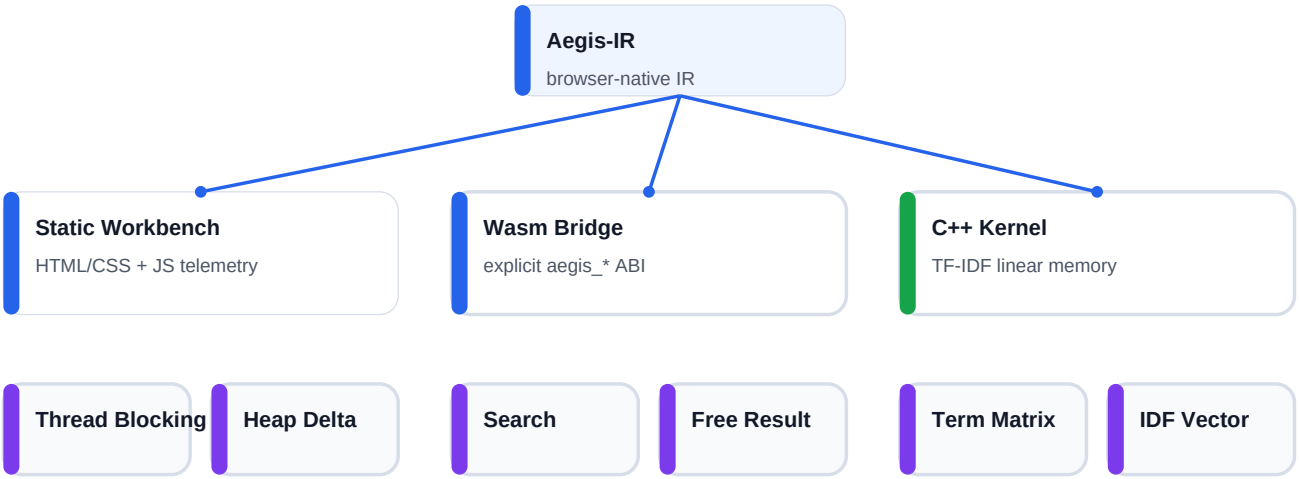
DESIGN REQUIREMENT

- Keep allocation-heavy scoring outside the JavaScript object heap.
- Represent term statistics as contiguous numeric buffers.
- Expose a minimal bridge so JavaScript orchestrates without owning the ranking loop.
- Measure main-thread blocking rather than reporting generic execution time alone.

AEGIS-IR LINEAR MEMORY ARCHITECTURE

Aegis-IR compiles an object-oriented C++ retrieval kernel to WebAssembly. The browser loads the module as a static asset, then invokes a small ABI for search, document count, memory telemetry, and result release. The ranking kernel owns term-frequency and inverse-document-frequency arrays inside linear memory.

SYSTEM TREE



CONTINUOUS ARRAY TF-IDF MODEL

Memory layout

term_frequencies[document_index * vocabulary_size + term_index]

```
tf(t,d) = 1 + ln(f(t,d))
idf(t) = ln((1 + N) / (1 + df(t))) + 1
S(q,d) = sum over t in T(q) of tf(t,d) * idf(t)
```

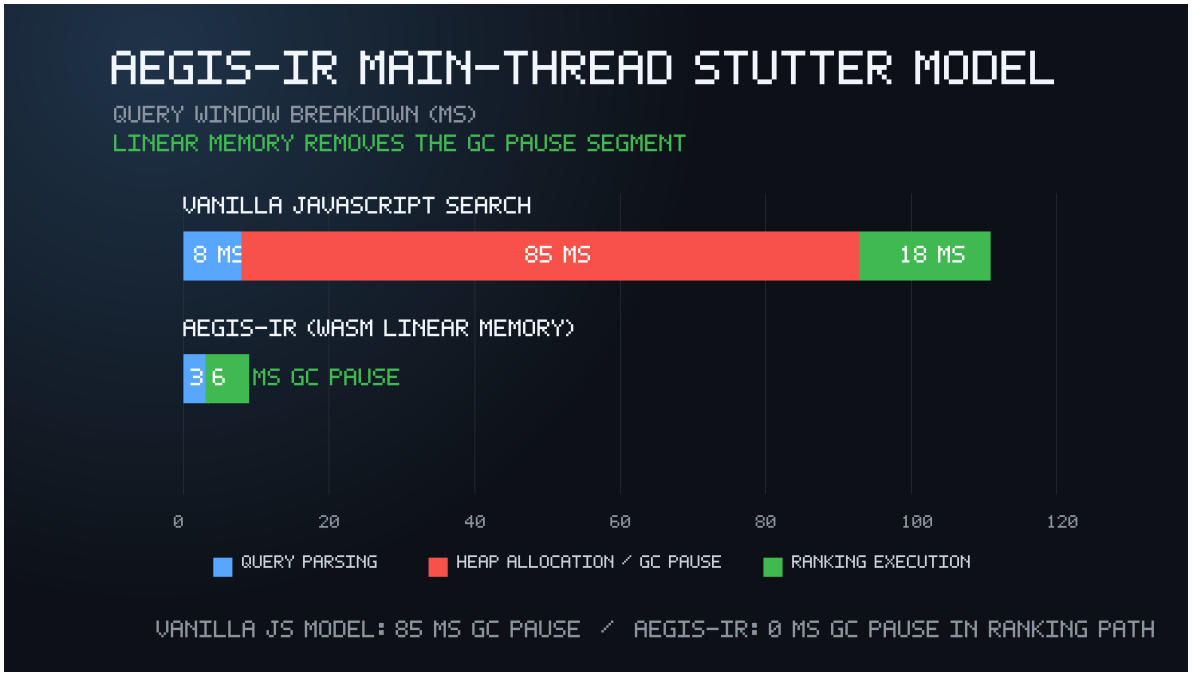
ABI SURFACE

Export	Return	Purpose
aegis_search(query)	char*	Executes ranking and returns compact payload
aegis_document_count()	int	Reports indexed corpus size
aegis_linear_memory_bytes()	int	Reports deterministic buffer footprint
aegis_free_result(ptr)	void	Releases native result memory explicitly

MAIN-THREAD TELEMETRY

Aegis-IR instruments the browser workbench around the user-visible failure mode: thread blocking. The JavaScript bridge records Long Task entries when supported, falls back to synchronous query-window duration, and reports linear-memory footprint from the Wasm engine. This separates ranking execution from garbage-collection pressure and DOM rendering.

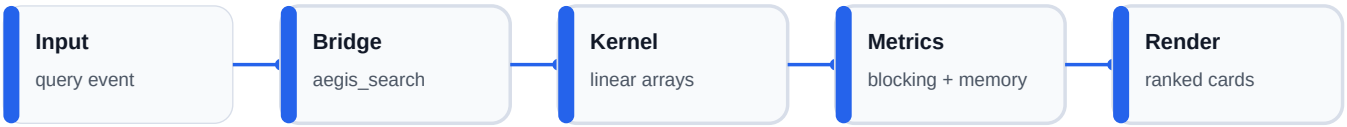
BENCHMARK VISUALIZATION



EXECUTION SEGMENT MODEL

Execution Segment	Vanilla JS	Aegis-IR	Interpretation
Query Parsing	8 ms	3 ms	smaller bridge work
Heap GC Pause	85 ms	0 ms	ranking avoids JS heap churn
Ranking	18 ms	6 ms	continuous numeric arrays
Total	111 ms	9 ms	reduced stutter window

TELEMETRY PIPELINE



DISCUSSION

Aegis-IR is best understood as a memory architecture for browser-native retrieval. The system does not argue that JavaScript is unsuitable for interfaces. It argues that allocation-heavy ranking loops should not be forced through the same managed object heap that must also support input, rendering, and layout. Moving TF-IDF scoring into WebAssembly linear memory creates a tighter and more predictable execution surface.

LIMITATIONS

- The current corpus is intentionally small and embedded for repeatable static deployment.
- The result payload still crosses into JavaScript for rendering; future work can use typed result arenas.
- Long Task availability differs across browsers, so telemetry should include browser/version metadata.
- The 0 ms GC segment describes the ranking path, not the entire page lifecycle.

FUTURE WORK

- Arena-based result buffers with offset tables instead of JSON serialization.
- BM25 and hybrid lexical ranking using the same deterministic memory discipline.
- Corpus ingestion pipeline with bounded allocation and offline persistence.
- Cross-browser stutter studies across V8, SpiderMonkey, and JavaScriptCore.

CONCLUSION

Aegis-IR demonstrates that browser-native search can be treated as a systems problem in deterministic memory design. By placing the TF-IDF ranking loop in C++ WebAssembly linear memory, the architecture removes the allocation-heavy scoring path from V8's managed heap and directly targets garbage-collection-induced main-thread stutter. The result is a zero-backend research workbench that is static-deployable, measurable, and aligned with the responsiveness expectations of modern browser products.

REFERENCES

[1] Mozilla Developer Network, WebAssembly Concepts and JavaScript Execution Model.
[2] W3C, Long Tasks API: Cooperative Scheduling of Background Tasks.
[3] Google V8 Project, Design notes on garbage collection and heap management.
[4] G. Salton and C. Buckley, Term-weighting approaches in automatic text retrieval.
[5] Emscripten Project Documentation, C/C++ to WebAssembly compilation pipeline.